QuickWheels

Vehicle Rental Management System

A Python OOP Case Study · Build-It-Yourself Specification

Level: Medium	Python 3.8+	No external packages	Spec only — students write the code

1. Case Description

QuickWheels rents bikes, cars and SUVs by the day. Build a menu-driven console application that manages Vehicles, Customers and Rentals with full CRUD operations, a dictionary datastore that simulates a database, and a unittest suite. Every OOP concept from the course must appear — the mapping is shown in *italics* throughout this specification.

2. Folder Structure

```
quickwheels_rental/
├── models/
│   ├── person.py
│   ├── customer.py
│   ├── vehicle.py
│   └── rental.py
├── services/
│   ├── customer_service.py
│   ├── vehicle_service.py
│   ├── rental_service.py
│   └── rental_system.py
├── data/
│   ├── datastore.py
│   └── seed_data.py
├── utils/
│   └── validator.py
├── tests/
│   ├── test_customer.py
│   ├── test_vehicle.py
│   └── test_rental.py
└── main.py
```

Keep an empty __init__.py in every package folder.

3. Models & Data Members (models/)

Class (file)	Data members	Behaviour & OOP concepts
Person · person.py	`person_id, name, age, phone`	Base class shared by people in the system. *(inheritance root)*
Customer(Person) · customer.py	`license_no`, private `__wallet`	`add_money()`, `pay()` → True/False, read-only `balance` property. *(inheritance, encapsulation)*
Vehicle · vehicle.py	Class attrs: `company="QuickWheels"`, `id_prefix`, `deposit_days`, auto-ID counter Instance: `vehicle_id, name, available, daily_rate` (`@property`, must be > 0)	`rental_cost(days)`, `__eq__`, `__lt__`, `__str__` *(class attributes, encapsulation, operator overloading — __eq__ powers v1 == v2, __lt__ powers the sorted vehicle view)*
Rental · rental.py	`rental_id` (class counter from 5001), `customer, vehicle, planned_days, amount_paid, deposit, status` ACTIVE/CLOSED	Invoice `__str__` · `__del__` prints `[ARCHIVE] Rental #id closed` · optional `damage_note` attached at return via `setattr` *(constructor, class attribute, destructor, dynamic attributes)*

Vehicle subclasses — each one overrides both the `rental_cost()` method and the `deposit_days` / `id_prefix` class attributes:

Subclass	id_prefix	deposit_days	rental_cost(days)
Bike	`"B-"`	`1`	`rate × days`
Car	`"C-"`	`2`	`rate × days × 1.05`
SUV	`"S-"`	`3`	`rate × days + 500`

(inheritance, method + class-attribute overriding)

4. Services & Utilities

File	Responsibility
`customer_service.py`	CRUD: add, get, list, update phone, delete.
`vehicle_service.py`	CRUD + `apply_festival_offer(id, pct)` using `setattr`; billing reads it with `getattr(v, "festival_offer", 0)`. *(dynamic attributes)*
`rental_service.py`	`rent(cust_id, veh_id, days)`: validate eligibility & availability, charge cost + deposit (`daily_rate × deposit_days`) from the wallet, mark vehicle unavailable. `return_vehicle(rental_id, actual_days, damage_note=None)`: late fee = extra days × rate × 1.5, damage fine ₹1000, refund `max(0, deposit − charges)`, reopen vehicle, status CLOSED. Plus get, list, delete.
`rental_system.py`	Orchestrator — the menu talks only to this class, never to the datastore. *(real-project layering)*
`utils/validator.py`	`is_eligible` (age ≥ 18 and has license), `is_available`, `positive_int`.

5. Dictionary Datastore (data/datastore.py)

```
DATASTORE = {
    "customers": {},   # "CU01"  -> Customer
    "vehicles":  {},   # "B-101" -> Bike / Car / SUV
    "rentals":   {}    #  5001   -> Rental
}
```

6. Sample Dataset (data/seed_data.py)

ID	Vehicle	Type	Daily rate
B-101	Honda Activa	Bike	₹400
C-201	Maruti Swift	Car	₹1,800
S-301	Toyota Innova	SUV	₹3,500

ID	Customer	Age	License	Wallet
CU01	Arjun	24	TN10-2021-0042	₹5,000
CU02	Meera	31	TN22-2018-0913	₹15,000
CU03	Rahul	17	(none)	₹2,000

IMPORTANT

7. Menu-Driven Interface (main.py)

Run inside a `while True` loop; invalid choices print a friendly message and re-show the menu.

```
====== QUICKWHEELS VEHICLE RENTAL ======
 1. Register customer
 2. Add vehicle
 3. View vehicles (sorted by rate)
 4. Rent a vehicle
 5. Return a vehicle
 6. View rentals
 7. Exit (archive rentals -> destructors fire)
```

NOTE

8. Test Cases — Two per File

Framework: `unittest` (standard library). Every test file's `setUp()` clears the datastore so each test starts from a fresh database — a key testing habit.

File	Test case	Expected result
`tests/test_customer.py`	`test_add_customer`	Added customer's ID appears in DATASTORE["customers"].
	`test_wallet_encapsulation`	pay() beyond balance returns False with balance unchanged; reading c.__wallet raises AttributeError.
`tests/test_vehicle.py`	`test_rental_cost_overriding`	Bike(400).rental_cost(2)==800 · Car(1800).rental_cost(2)==3780 · SUV(3500).rental_cost(2)==7500.
	`test_rate_validation`	Setting daily_rate to 0 or negative raises ValueError.
`tests/test_rental.py`	`test_rent_success`	Rental stored, vehicle available == False, wallet debited exactly cost + deposit.
	`test_ineligible_blocked`	Renting as Rahul is refused and nothing is stored.

9. Run Instructions

Always run from the project root:

```
python main.py                          # app (seed data auto-loads)
python -m unittest discover tests -v    # all tests
```

STRETCH (OPTIONAL)